

Agent Support Chat System

Complete Technical Architecture & Flow Documentation

Covers WebSocket (Django Channels), Redis message storage, REST APIs, authentication, Admin ↔ Agent chat flow, General & Application-specific chats, and known issues.

Prepared from actual project source code | Django + React (Vite) + Channels + Redis

1 System Overview

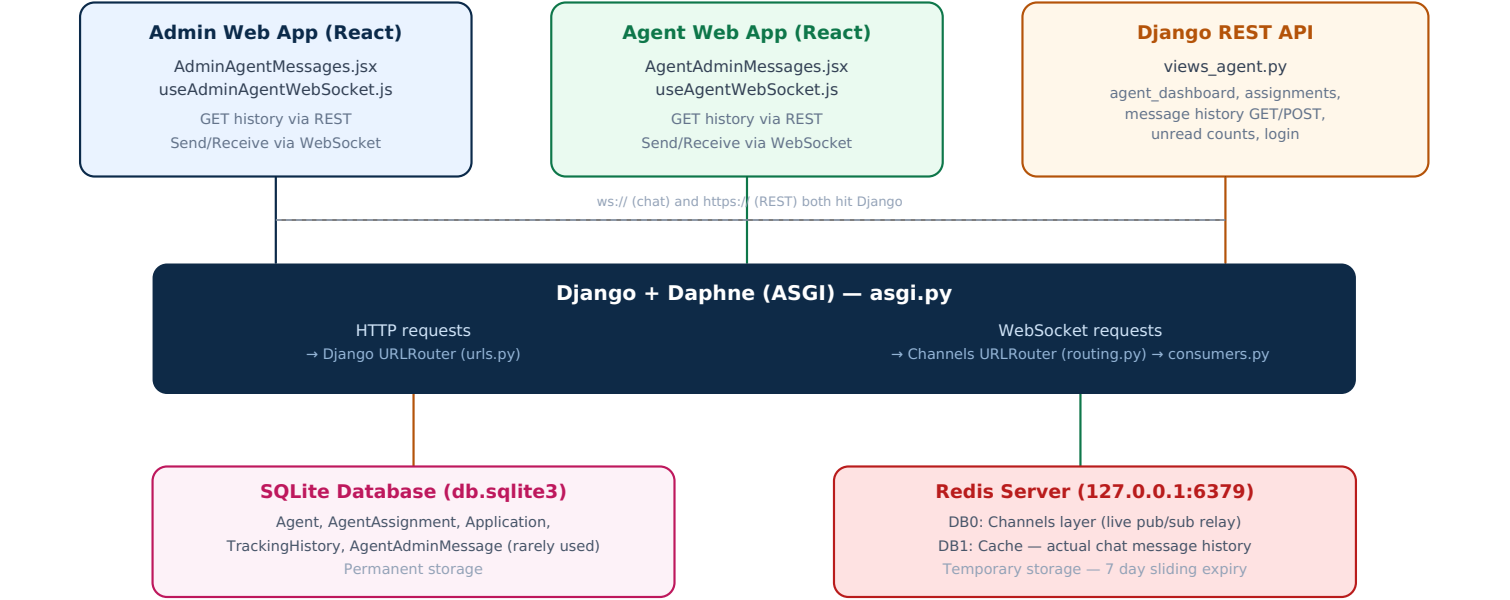
This document explains, in plain language, how the **Agent Support Chat** feature of the 100 Transcripts platform works end-to-end: how an Admin talks to a Field Agent, how each conversation is kept separate, where messages are actually stored, and how long they last. It is based directly on your project files (`consumers.py` , `redis_chat.py` , `views_agent.py` , `models.py` , `settings.py` , `urls.py` , the WebSocket hooks, and the chat page components).

In one sentence: Chat messages travel over a **WebSocket** connection managed by **Django Channels**, get temporarily stored in **Redis** (not the SQL database) for 7 days, and are broadcast live to everyone connected to that specific Admin↔Agent "room".

1.1 Two Kinds of Conversations

Chat Type	When it's used	Redis Key Pattern
Application Chat	Admin discussing one specific student application with the agent handling it	<code>chat:agent:<agent_id>:app:<application_id></code>
General Chat	Queries not tied to any specific application	<code>chat:agent:<agent_id>:general</code>

2 High-Level Architecture



Both the Admin and Agent apps talk to the **same** WebSocket consumer and the **same** REST views — the system tells them apart using the `agent_id` and `application_id` in the URL, plus the `is_from_admin` flag inside each message.

3 Key Files & Their Responsibility

3.1 Backend (Django) — backend/Transcripts/App/

File	Responsibility
consumers.py	The WebSocket "brain". Defines <code>AdminAgentConsumer</code> — handles connect, disconnect, receiving a chat message, saving it, and broadcasting it to everyone in that chat room.
redis_chat.py	All Redis read/write logic for chat history: builds the Redis key, appends a new message, trims to last 1000 messages, and sets/refreshes the 7-day expiry.
routing.py	Maps WebSocket URL patterns (e.g. <code>/ws/agent/<agent_id>/app/<app_id>/</code> and <code>/ws/agent/<agent_id>/general/</code>) to <code>AdminAgentConsumer</code> .
views_agent.py	All Agent-module REST endpoints: agent login/token auth, admin agent CRUD, assignment workflow (accept/reject/status), dashboard, and the chat REST endpoints (history GET, DB-based POST — see §6.2).
models.py	Database tables: <code>Agent</code> , <code>AgentAssignment</code> , <code>Application</code> , <code>TrackingHistory</code> , <code>AgentAdminMessage</code> , plus the wider platform models (Users, College, Payment, Document, Issue, etc.).
urls.py	Maps every REST URL (<code>/api/...</code>) to its view function.
Transcripts/settings.py	Configures Channels (Redis DB0) and Django cache (Redis DB1), installed apps, CORS, database engine (SQLite).
Transcripts/asgi.py	Wires HTTP and WebSocket protocols together for the Daphne ASGI server.

3.2 Frontend (React) — frontend/src/

File	Responsibility
Admin/pages/AdminAgentMessages.jsx	Admin's chat screen — conversation list (left) + chat window (right). Loads history via REST, sends/receives live via WebSocket.
Admin/hooks/useAdminAgentWebSocket.js	Opens/manages the Admin side WebSocket connection and exposes <code>sendMessage()</code> .
Admin/agent/pages/AgentAdminMessages.jsx	Agent's own chat screen (mirror of the admin one), scoped to the logged-in agent only.
Admin/agent/hooks/useAgentWebSocket.js	Opens/manages the Agent side WebSocket connection and exposes <code>sendMessage()</code> .
Admin/agent/pages/AgentLogin.jsx	Agent login form — stores the signed auth token and agent profile in <code>localStorage</code> .

Duplicate / unused files found: `Admin/agent/hooks/useAdminWebsocket.js` is a near-identical copy of `useAdminAgentWebSocket.js` but is not imported by any page shown in the project — it looks like an earlier draft left behind. Recommend deleting it (or confirming nothing else references it) to avoid confusion during future edits.

4 Step-by-Step Message Flow

4.1 What happens when Admin sends a message

- Admin opens a conversation → `AdminAgentMessages.jsx` works out the `agent_id` and `appId` from the URL / conversation list.
- `useAdminAgentWebSocket` opens: `ws://127.0.0.1:8000/ws/agent/<agent_id>/app/<appId>/` (or `/general/`).
- Django Channels routes this to `AdminAgentConsumer.connect()`, which joins the socket to a "room" named `agent_<id>_app_<appId>` or `agent_<id>_general`.

- 4. Admin types a message and presses Send → `sendMessage(text, applicationId, true)` sends `{ "message": "...", "application_id": 12, "is_from_admin": true }` over the socket.
- 5. `AdminAgentConsumer.receive()` on the server parses this JSON and calls `redis_chat.save_message(agent_id, application_id, message, is_from_admin=True)` .
- 6. `save_message()` appends the message (with a new UUID and timestamp) to the Redis list for that room's key, trims it to the last 1000 messages, and resets the 7-day expiry timer on that key.
- 7. The consumer then calls `channel_layer.group_send()` , which broadcasts the message to **every** socket currently joined to that same room (this is the Channels/Redis pub-sub layer, DB0).
- 8. Every connected browser tab in that room (the Admin's own tab, and the Agent's tab if they are online) receives a `chat_message` event and appends it to the on-screen chat instantly — no page refresh needed.

4.2 What happens when an Agent sends a message

Identical mechanism, mirrored: the Agent's `useAgentWebSocket` hook connects to the exact same room URL pattern, and `sendMessage(text, applicationId, false)` sends `is_from_admin: false` . The server-side handling in `consumers.py` does not care which side sent it — it treats Admin and Agent symmetrically and stores/broadcasts the message the same way.

4.3 How multiple Agents stay isolated from each other

Isolation is achieved purely through the **room name**, which always embeds the specific `agent_id : agent_<id>_app_<appId> / agent_<id>_general` . Agent A can never receive Agent B's messages because their sockets are joined to entirely different Channels groups. The same is true for Redis storage — each agent+conversation combination gets its own, separate key, so histories never mix.

4.4 General Chat vs Application Chat — how the code tells them apart

	Application Chat	General Chat
URL param	<code>app_id</code> present	<code>app_id</code> absent
Room name	<code>agent_<id>_app_<appId></code>	<code>agent_<id>_general</code>
Redis key	<code>chat:agent:<id>:app:<appId></code>	<code>chat:agent:<id>:general</code>
Frontend identifier	numeric application id	the literal string <code>"general"</code>

5 WebSocket, Django Channels & Redis — In Detail

5.1 How the WebSocket connects

1. Browser opens a `ws://` connection to Daphne (the ASGI server running Django).
2. `asgi.py` routes it into the Channels `URLRouter` (defined in `routing.py`), which matches the URL pattern and hands it to `AdminAgentConsumer`.
3. `connect()` reads `agent_id` and `app_id` from the URL, builds the room name, calls `channel_layer.group_add()` to join that room, then `accept()`s the connection and sends a small `{"type": "connection", ...}` confirmation back to the browser.

5.2 Two completely different Redis roles

Redis Database	Used by	Purpose	Persistent?
DB 0 (default)	<code>CHANNEL_LAYERS</code> in <code>settings.py</code> (<code>channels_redis</code>)	Real-time delivery only — relays a message from one socket to all others in the same room at the instant it is sent.	No — nothing is kept here after delivery.
DB 1	<code>CACHES["default"]</code> in <code>settings.py</code> (django Redis cache), used via <code>redis_chat.py</code>	Actual chat history — the list of past messages per conversation, so history loads when you open a chat.	Yes, but only for 7 days (see 5.3).

5.3 How the 7-day expiry actually works

Every time `save_message()` runs, it does `cache.set(key, messages, MESSAGE_TTL)` where `MESSAGE_TTL = 60 * 60 * 24 * 7` seconds. This **resets the countdown to 7 full days from that moment**, for that specific conversation's key only.

- An **active** conversation (messages sent regularly) effectively never expires, because every new message pushes the clock forward again.
- An **inactive** conversation with no new messages for 7 straight days will have its entire history silently deleted by Redis — there is no warning, backup, or partial expiry.
- Expiry is **per conversation**, not global — a busy application chat and a quiet general chat with the same agent expire independently.

Message list capping: each key is also trimmed to the last **1000 messages** (`messages[-MAX_MESSAGES:]`) every time a new one is saved — older messages beyond that are dropped, even inside the 7-day window.

5.4 What is stored in the SQL Database vs Redis

Data	Where	Notes
Agent profile, credentials, workload	SQLite (<code>Agent</code> model)	Permanent
Assignment status / workflow (accepted, in progress, delivered...)	SQLite (<code>AgentAssignment</code>)	Permanent, drives the whole agent workflow
Status-change audit trail ("Recent Activity")	SQLite (<code>TrackingHistory</code>)	Permanent — separate from chat
Live chat message text	Redis DB1 only	Not written to SQLite by the real-time flow — see §6.2 for the exception
<code>AgentAdminMessage</code> DB table	SQLite	Exists in <code>models.py</code> , but is only populated by an HTTP code path the current UI does not actually call (dead path) — see §6.2

6 APIs & Authentication

6.1 Relevant REST endpoints (from `urls.py`)

Endpoint	Purpose
<code>POST /api/agent/login/</code>	Agent logs in with email + password, receives a signed token
<code>GET /api/admin/agent-support/all-conversations/</code>	Admin's conversation list (per-application + general)
<code>GET/POST /api/admin/applications/<app_id>/messages/</code>	Admin: load / (legacy) post chat history for one application
<code>GET/POST /api/admin/agent-support/general/<agent_id>/</code>	Admin: load / (legacy) post general chat history
<code>GET/POST /api/agent/<agent_id>/admin-messages/<app_id>/</code>	Agent: load / (legacy) post chat history for one application
<code>GET/POST /api/agent/<agent_id>/admin-messages/general/</code>	Agent: load / (legacy) post general chat history
<code>GET /api/agent/<agent_id>/admin-messages/unread-count/</code>	Unread badge count for the agent
<code>GET /api/admin/agent-support/unread-count/</code>	Unread badge count for the admin
<code>GET /api/agent/<agent_id>/dashboard/</code>	Agent home screen: stats, today's tasks, visits, deliveries, recent activity

6.2 Important inconsistency: two different "send" paths exist

The chat feature currently has two parallel, disconnected code paths:

1. **WebSocket path (the one the UI actually uses to send messages):** `consumers.py` → `redis_chat.save_message()`. Stores only in **Redis**. This is what you see live in the app.
2. **HTTP POST path (present in `views_agent.py`, but not called by the current "Send" buttons):** `admin_agent_messages()` /

`agent_admin_messages()` POST handlers create an `AgentAdminMessage` row directly in the **SQL database** — completely separate from Redis.

Because history is always loaded from Redis (`get_messages()`), any row created via path 2 will **never show up** in the chat history the UI displays, and will not be included in the 7-day Redis expiry logic at all. In addition, on the admin side, the POST handler broadcasts to a Channels group named `agent_<agent.id>` — which does not match the room naming pattern (`agent_<id>_app_<appId>` / `agent_<id>_general`) used everywhere else, so nobody is actually listening on that group. This code path is effectively dead / broken.

6.3 Authentication

Layer	Mechanism	Status
Agent-side REST endpoints (<code>@agent_required</code>)	Signed, stateless token from <code>django.core.signing</code> (7-day expiry), sent as <code>Authorization: Bearer <token></code>	Implemented correctly in the backend
Admin-side agent-support REST endpoints	No server-side check at all in <code>views_agent.py</code> for these views (e.g. <code>admin_agent_messages</code> , <code>admin_all_conversations</code>)	Open / unauthenticated on the backend
WebSocket (<code>AdminAgentConsumer</code>)	None — <code>connect()</code> accepts any <code>agent_id</code> / <code>app_id</code> combination with no token check	Open — anyone who knows/guesses an <code>agent_id</code> can join and read/send in that chat room

Critical bug found — Agent chat is likely broken end-to-end for regular users:

`AgentLogin.jsx` saves the login token as `localStorage.setItem("agentToken", ...)` (camelCase), but `AgentAdminMessages.jsx` (and other agent pages) read it back as `localStorage.getItem("agent_token")` (snake_case). These two keys never match, so the token read by the chat page is always `null`. Every protected REST call the Agent chat page makes (`fetchAssignments`, `fetchMessages`, `fetchUnreadCount`) will fail with `401 AUTH_REQUIRED`, and the chat screen will show its error state instead of any messages.

Fix: use the same key name in both files — e.g. always `localStorage.setItem("agent_token", ...)` on login.

7 Current State: What Works, What's Risky

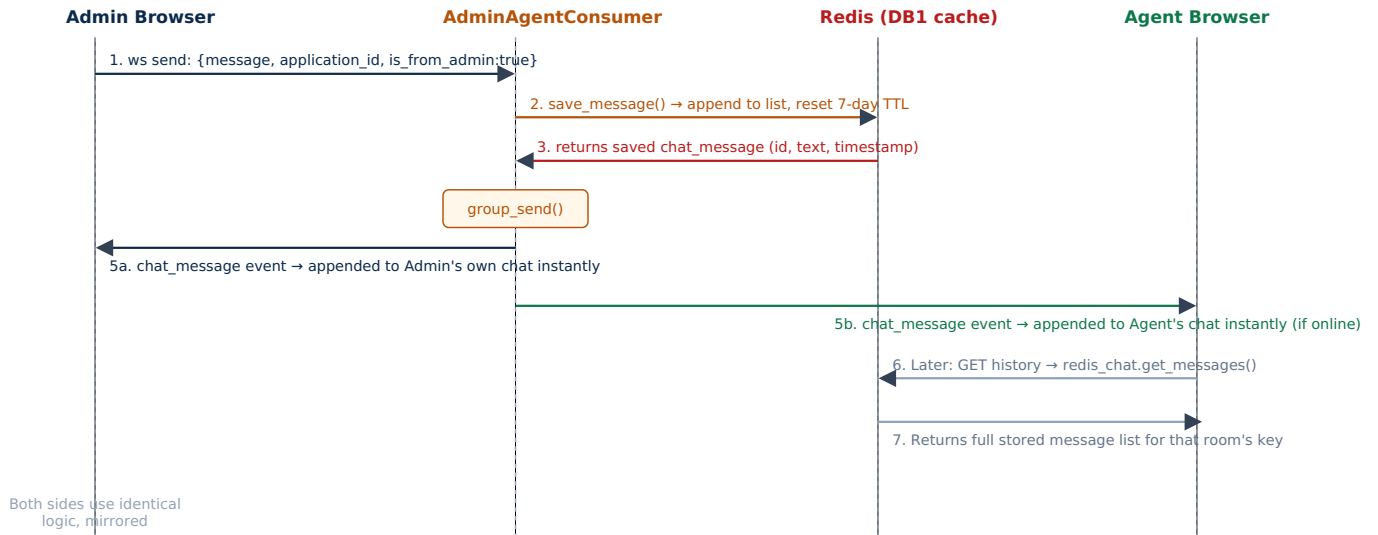
7.1 Working today

- **Live, real-time messaging over WebSocket between one Admin tab and one Agent tab (once the agent token bug above is fixed).**
- **Per-agent and per-application chat room isolation (no cross-talk between different agents/applications).**
- **7-day rolling history with automatic trimming to 1000 messages per conversation.**
- **Agent login & signed token generation/verification on the backend.**
- **Admin conversation list assembled from live `AgentAssignment` + DB message rows.**

7.2 Risks & problems found

#	Issue	Impact	Severity
1	Agent login stores token as <code>agentToken</code> , chat page reads <code>agent_token</code>	Agent chat / dashboard fetches fail with 401	High
2	WebSocket consumer has no authentication at all	Anyone who can reach the server can connect to any agent's chat room by guessing an id	High
3	Admin agent-support REST endpoints have no auth check	Chat history, conversation lists, and messages are readable/writable without login	High
4	Two disconnected "send message" code paths (WS→Redis vs HTTP POST→DB)	DB-based path is dead code; unread counts (which read the DB table) don't reflect real WS-sent messages	Medium
5	Admin POST handler broadcasts to a Channels group name nobody joins	That POST path silently does nothing in real time even if it were called	Medium
6	File attachments: UI lets you pick a file, but "Send" only transmits text over WebSocket	Attachment upload from the chat box does not actually work yet	Medium
7	Passwords for Agent / Admin / Users models stored as plain text in the database	Any DB access exposes real passwords, no hashing	High
8	<code>DEBUG=True</code> , <code>ALLOWED_HOSTS=['*']</code> , <code>CORS_ALLOW_ALL_ORIGINS=True</code> , hardcoded <code>SECRET_KEY</code> committed in <code>settings.py</code>	Not safe for a production deployment as-is	Medium (fine for dev, must fix before going live)
9	Duplicate hook file <code>useAdminWebSocket.js</code> unused, near-duplicate of <code>useAdminAgentWebSocket.js</code>	Dead code, maintenance confusion risk	Low
10	<code>is_read</code> is hard-coded <code>true</code> in every history response; never actually toggled based on real read state	Read receipts / unread badges are not fully trustworthy	Low-Medium
11	Redis history has no backup — 7 days of silence permanently deletes the whole conversation	Long-idle conversations lose history with no recovery option	Medium

8 Message Send/Receive Sequence (Visual)



9 Checklist

9.1 Completed / working

- ☒ Real-time WebSocket chat between Admin and Agent (Django Channels)
- ☒ Redis-backed chat history with 7-day rolling expiry per conversation
- ☒ Per-agent, per-application room isolation
- ☒ General chat vs application-specific chat separation
- ☒ Agent signed-token login/authentication mechanism (backend)
- ☒ Admin conversation list combining assignments + general chats
- ☒ Unread-count endpoints (backend logic present, though data source is inconsistent — see 9.2)

9.2 Pending / needs fixing

- ☐ Fix the `agentToken` vs `agent_token` localStorage key mismatch (Agent chat is broken without this)
- ☐ Add authentication to the WebSocket consumer (`connect()` currently trusts the URL blindly)
- ☐ Add authentication to the admin agent-support REST endpoints
- ☐ Remove or fix the dead HTTP POST "send message" path (DB write with no matching broadcast group)
- ☐ Decide on one single source of truth for chat messages (Redis vs DB) and make unread counts read from it
- ☐ Wire up real file-attachment sending through the WebSocket flow (or a matching REST upload call)
- ☐ Hash Agent/Admin/User passwords instead of storing plain text
- ☐ Turn off `DEBUG`, restrict `ALLOWED_HOSTS` and CORS, move `SECRET_KEY` to an environment variable before production
- ☐ Remove unused duplicate hook file `useAdminWebSocket.js`
- ☐ Implement real `is_read` tracking instead of hard-coded `true`

9.3 Should be tested

- ☐ Agent login → open chat → confirm messages actually load (currently likely fails, see High-severity bug)
- ☐ Two browser tabs (Admin + Agent) on the same application chat — confirm both sides see messages live
- ☐ Leave a conversation untouched for 7+ days in a staging environment and confirm expected history loss
- ☐ Try connecting to another agent's chat room by manually editing the `agent_id` in the WebSocket URL, to confirm the isolation/authorization gap
- ☐ Send 1000+ messages in one room and confirm oldest messages are correctly trimmed
- ☐ Confirm general chat and application chat for the same agent never mix messages
- ☐ Load test: multiple agents chatting simultaneously, confirm no cross-room leakage